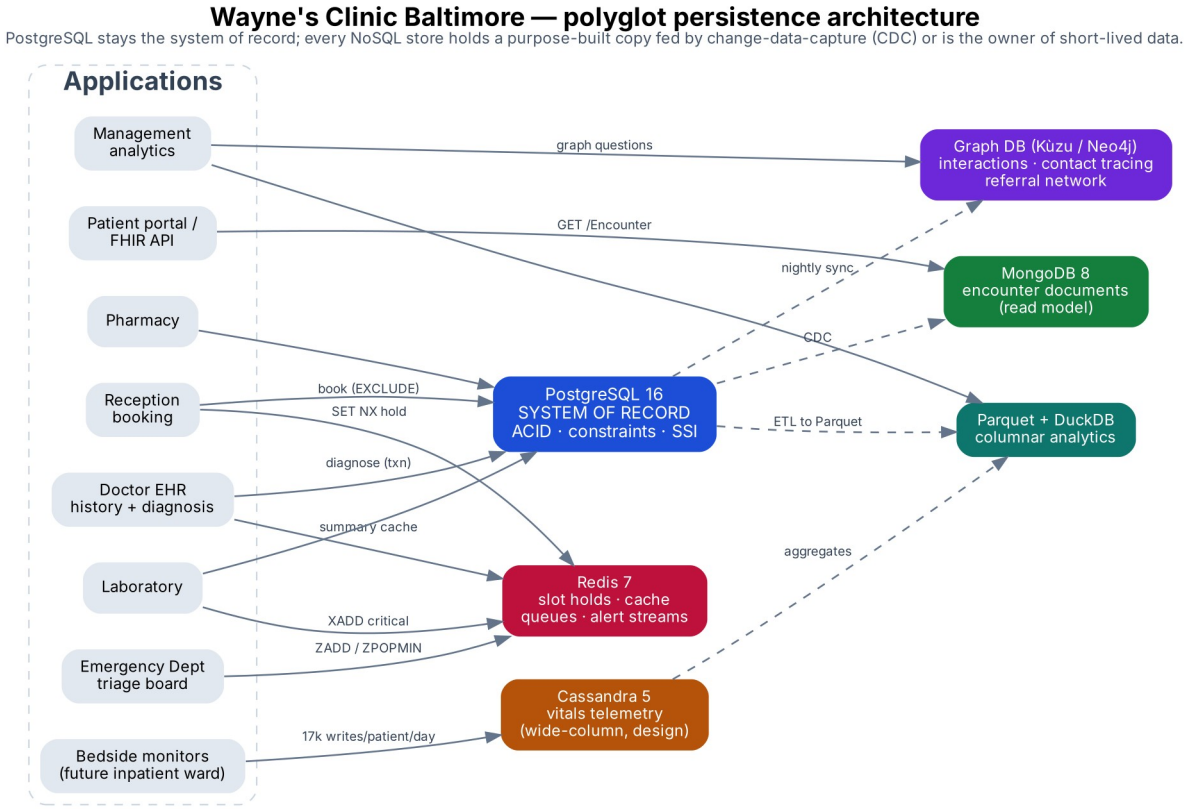
Wayne's Clinic Baltimore - Advanced Database Systems

Portfolio 4 — NoSQL and Advanced Data Models

Organisation: Wayne's Clinic Baltimore · **Engines:** MongoDB 8.0.23 · Redis 7 · Kùzu 0.11 (Cypher property graph) · DuckDB + Parquet · Apache Cassandra 5 (design)

The same clinic, the same data and the same three core operations, modelled four more ways. Every model except Cassandra was **built and executed**: the data was migrated from the Portfolio 1–2 PostgreSQL database, and the queries, explains and timings below are captured output. Cassandra is a query-first design only, because its binaries could not be downloaded in the build environment, and is labelled as such.

Model	Store	What it is used for here	Folder
Document	MongoDB 8 (replica set)	Encounter documents with embedded diagnoses, prescriptions and lab results; booking, diagnosis transaction, history	`document-mongodb/`
Key-value	Redis 7	Slot holds, cache-aside summary card, emergency triage queue, critical-result stream	`key-value-redis/`
Graph	Kùzu (Cypher) + Neo4j script	Drug interactions, contact tracing, referral network, comorbidity, shortest paths	`graph-cypher/`
Column	DuckDB on Parquet (executed); Cassandra CQL (design)	Columnar analytics; wide-column booking, timeline and telemetry tables	`columnar-duckdb/`, `column-family-cassandra/`



---

1. Document model — MongoDB

1.1 From tables to documents

A document database stores an aggregate, meaning the data that is read and written together, as one document. The modelling question is what to **embed** and what to **reference**:

Relational tables	Document design	Why
`doctor` + `doctor_availability` + `specialty` + `department`	one `doctors` document, availability **embedded**	Bounded (≤ 14 sessions), always read together when booking
`patient` + `patient_allergy` + `postal_area`	one `patients` document, allergies and address **embedded**	Owned by the patient, needed on every prescribing decision
`appointment` + `diagnosis` + `prescription` + `prescription_item` + `lab_order` + `lab_result` + `icd10_code`/`medication`/`lab_test` names	one `encounters` document per appointment, all clinical records **embedded**	The visit is written as a unit and read as a unit (history screen)
`appointment.patient_id`	`encounters.patientId` **reference** (+ copied name: extended-reference pattern)	Embedding every visit inside the patient would grow without limit (16 MB document cap) and make every clinician write to the same document

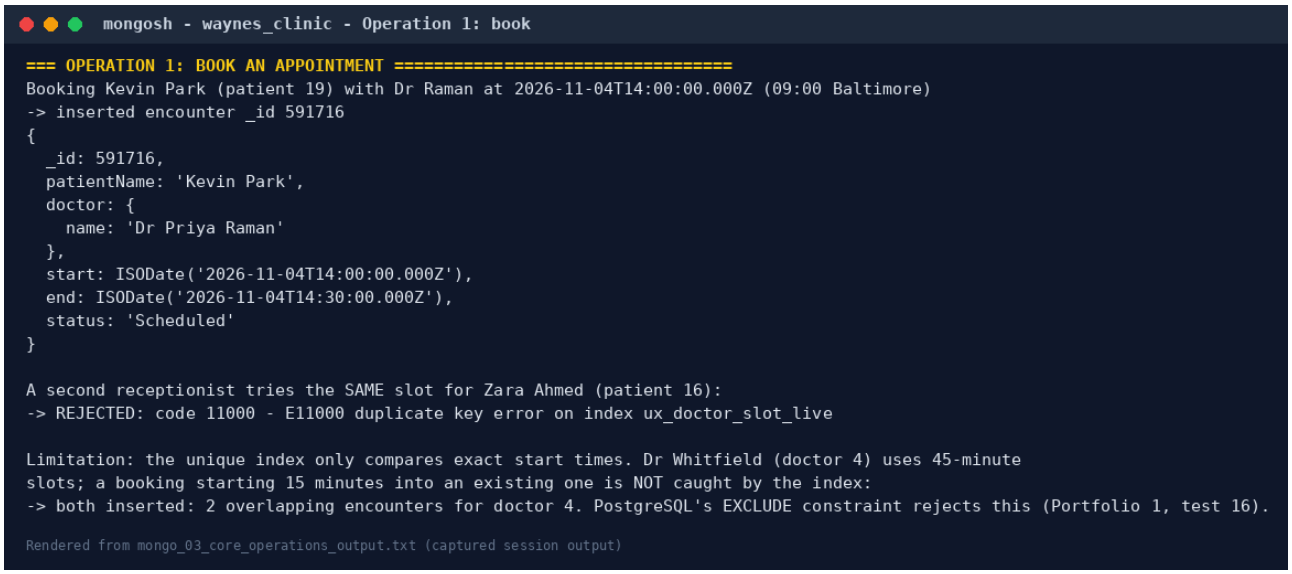
The migration (`02_load_from_postgres.py`) lets PostgreSQL assemble each document with `json_build_object`/`json_agg`, so the mapping is visible in one SQL statement per collection. It loaded **641,454 documents** in about 60 s: 200 doctors, 50,021 patients and 591,233 encounters. Example documents: `sample_encounter_document.json`, `sample_patient_document.json`.

"Schema-less" does not mean "no rules". Each collection has a `$jsonSchema` validator (`01_create_collections.js`) for MRN format, sex and blood group enums, the ICD-10 code pattern,

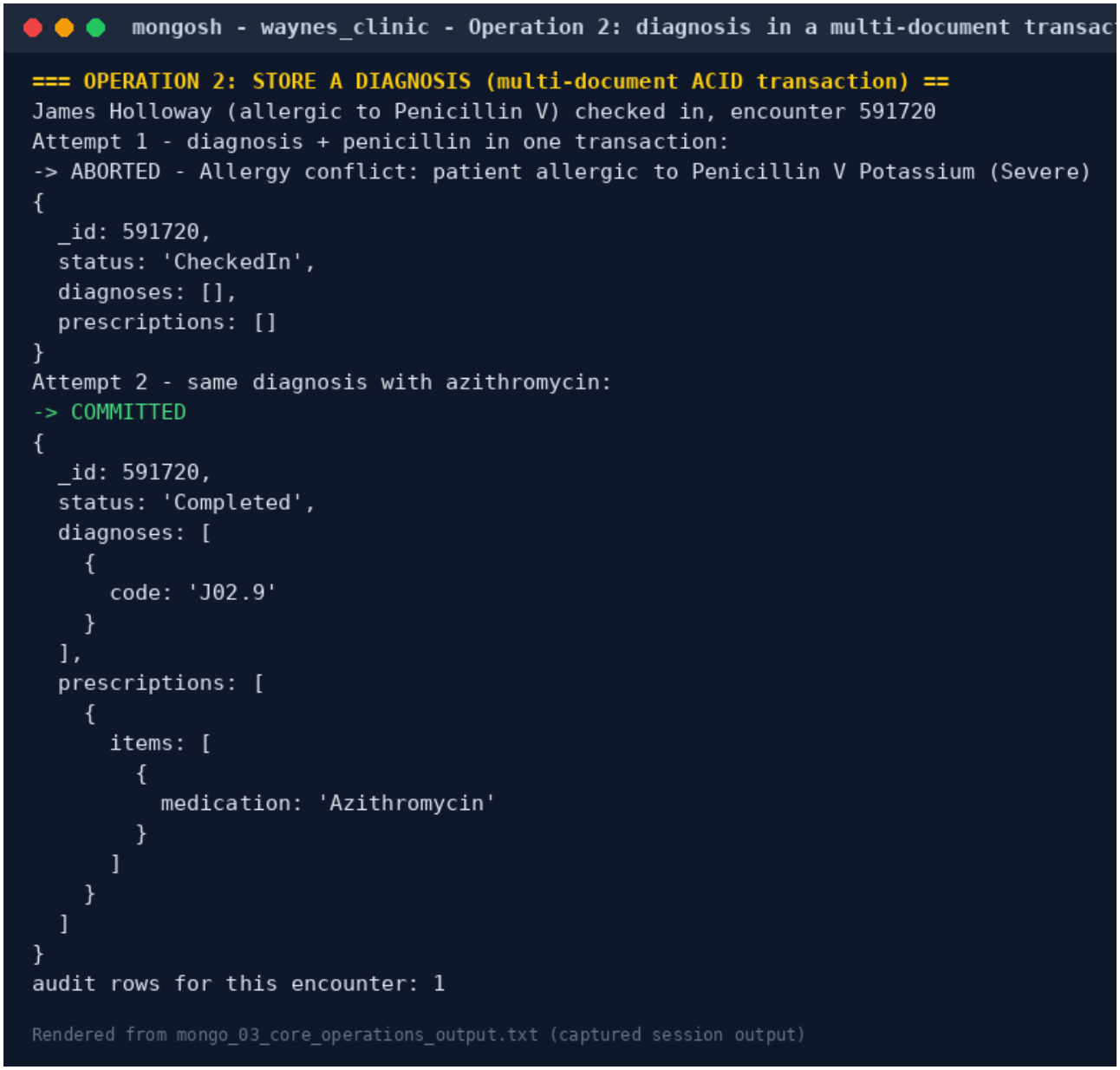
appointment status and required fields. Invalid inserts and updates are rejected with code 121 (`DocumentValidationFailure`).

1.2 The three core operations (`03_core_operations.js`)

Book. `bookAppointment()` checks the doctor's embedded availability in Baltimore time, takes an id from a counter document, and inserts. A **partial unique index** on `{doctor.id, start}` (live statuses only) rejects a second booking of the same slot with `E11000 duplicate key`. **Limitation, shown in the output:** the index compares only exact start times. A 45-minute slot booked at 13:30 and a second booking at 13:45 both succeed. MongoDB has no equivalent of PostgreSQL's `EXCLUDE … WITH &&` range constraint, so overlap rules depend on a fixed slot grid or application logic.

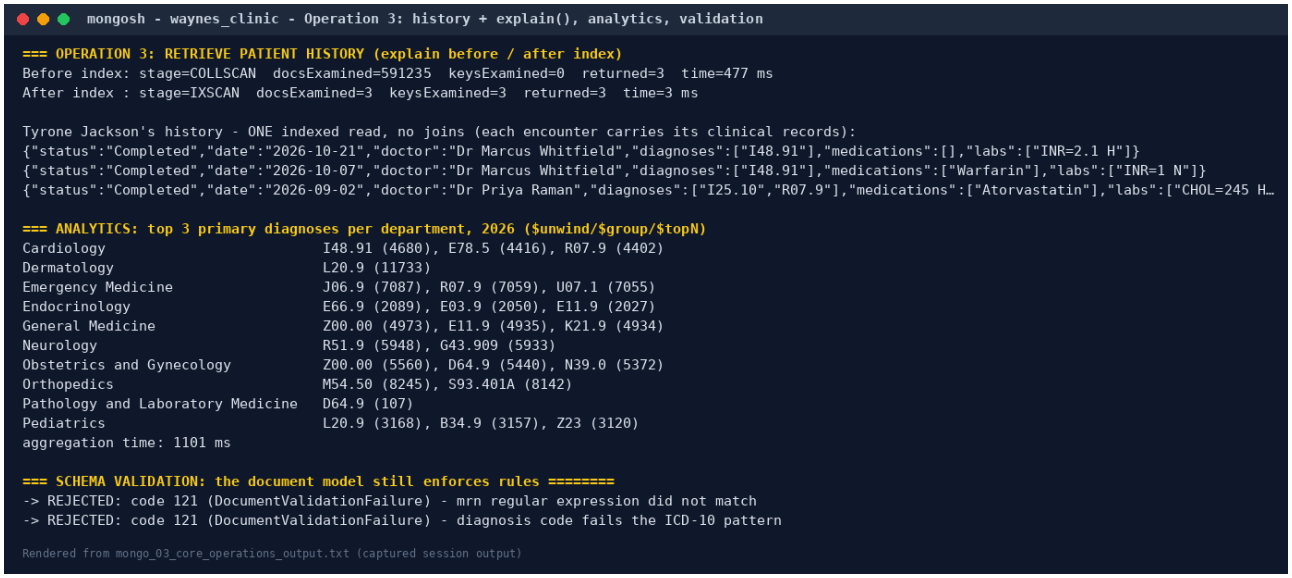


Store a diagnosis. A **multi-document ACID transaction** (`session.withTransaction`, `readConcern: snapshot`, `writeConcern: majority`) reads the encounter and the patient's allergies, updates the encounter (status, diagnoses, prescription) and writes an audit document. Prescribing penicillin to James Holloway aborts the whole transaction, leaving no diagnosis and no audit row. Azithromycin commits.



Retrieve history. One indexed query returns every visit with its clinical records already inside; no joins are needed.

	Plan	Documents examined	Time
Before index	COLLSCAN	591,235	477 ms
After `{patientId: 1, start: -1}`	IXSCAN	3	3 ms



1.3 Document vs relational: measured

Measure (same 300 random patients, warm cache)	PostgreSQL 4-branch history view	MongoDB `find` on encounters
Median latency	1.25 ms	**0.75 ms**
p95 latency	2.03 ms	**1.11 ms**
Storage, clinical data incl. indexes	487.6 MB	**154.0 MB** (WiredTiger compression; 435.7 MB uncompressed)
Top-3 diagnoses per department (2026 analytics)	**222 ms** (Portfolio 2, E6)	1,101 ms (`$unwind`/`$group`/`$topN`)

The document model wins the read it was designed for (about 1.7× faster here) and stores the data in a third of the space. It loses ad-hoc analytics, and the booking overlap rule had to be weakened.

2. Key-value model — Redis (`redis_demo.py`)

Redis is used **beside** PostgreSQL for jobs where a key-value store is the right tool; it is never the system of record. The full redis-cli transcript is in `evidence/redis_demo_output.txt`.

Use case	Redis structure and commands	Evidence
R1 Slot hold while a receptionist fills in the form	`SET slot:hold:doctor:1:… receptionist-A NX EX 300`. B's `NX` returns `(nil)`. Release uses a Lua compare-and-delete so only the owner can release; an abandoned hold expires on its own	atomic, self-expiring lock
R2 Cache-aside patient summary card	`HSET` + `EXPIRE 600`, invalidated on change	PostgreSQL 0.324 ms vs Redis 0.151 ms median (2×)
R3 Emergency triage queue	sorted set, score = triage level × 10^{10} + arrival time; `ZPOPMIN` pops the most urgent patient atomically	level-1 arrival at 19:12 is seen before level-3 arrivals from 19:02
R4 Critical-result alerts	Stream + consumer group: `XADD`, `XREADGROUP`, `XACK`, `XPENDING`	unacknowledged alerts survive a consumer crash
R5 Live dashboard counters	`HINCRBY`, `INCR`	atomic, no read-modify-write race

redis-cli - slot hold (SET NX EX) and cache-aside

```
R1  SLOT HOLD - a 5-minute lock while a receptionist completes a booking (SET NX EX)
================================================================================
# Receptionist A opens the booking form for Dr Okafor's 09:00 slot:
127.0.0.1:6379> SET slot:hold:doctor:1:2026-11-04T09:00 receptionist-A NX EX 300
OK
# Receptionist B opens the same slot a second later - NX refuses because the key exists:
127.0.0.1:6379> SET slot:hold:doctor:1:2026-11-04T09:00 receptionist-B NX EX 300
(nil)
127.0.0.1:6379> GET slot:hold:doctor:1:2026-11-04T09:00
"receptionist-A"
127.0.0.1:6379> TTL slot:hold:doctor:1:2026-11-04T09:00
(integer) 300
# B is told 'slot being booked by another user'. A completes the booking in PostgreSQL
# (the EXCLUDE constraint is still the final guard), then releases the hold - but only
# if A still owns it. A Lua script makes check-and-delete atomic:
127.0.0.1:6379> EVAL "if redis.call('GET', KEYS[1]) == ARGV[1] then return redis.call('DEL', KEYS[1]) else return 0 end" 1 slot:hold:doctor:1:2026-11…
(integer) 0
127.0.0.1:6379> EVAL "if redis.call('GET', KEYS[1]) == ARGV[1] then return redis.call('DEL', KEYS[1]) else return 0 end" 1 slot:hold:doctor:1:2026-11…
(integer) 1
127.0.0.1:6379> EXISTS slot:hold:doctor:1:2026-11-04T09:00
(integer) 0
# If A's browser crashes, the hold simply expires after 300 s - no stale lock remains.

================================================================================
R2  CACHE-ASIDE - patient summary card read on every screen
================================================================================
127.0.0.1:6379> GET patient:WCB0000001:summary
(nil)
# MISS -> read from PostgreSQL, then store as a hash with a 10-minute TTL:
127.0.0.1:6379> HSET patient:WCB0000001:summary mrn WCB0000001 name "James Holloway" dob 1958-03-14 blood_group O+ allergies "Penicillin V Potassium …
(integer) 7
127.0.0.1:6379> EXPIRE patient:WCB0000001:summary 600
(integer) 1
127.0.0.1:6379> HGETALL patient:WCB0000001:summary
1) "mrn"
2) "WCB0000001"
3) "name"
4) "James Holloway"
5) "dob"
6) "1958-03-14"
7) "blood_group"
8) "O+"
9) "allergies"
10) "Penicillin V Potassium (Severe)"
11) "problems"
12) "E11.9, I10, J02.9"
13) "last_visit"
14) "2026-10-25"
... (5 more lines in the .txt log)
```

Rendered from redis_demo_output.txt (captured session output)

```
redis-cli - triage queue (sorted set) and critical-result stream

R3  EMERGENCY DEPARTMENT TRIAGE QUEUE (sorted set: priority, then arrival time)
================================================================================
# score = triage_level * 10^10 + arrival epoch seconds  -> level 1 (resuscitation) always first
127.0.0.1:6379> ZADD ed:queue 31793210520 WCB0000011|triage3|19:02
(integer) 1
127.0.0.1:6379> ZADD ed:queue 41793210700 WCB0000019|triage4|19:05
(integer) 1
127.0.0.1:6379> ZADD ed:queue 21793210820 WCB0000003|triage2|19:07
(integer) 1
127.0.0.1:6379> ZADD ed:queue 31793210940 WCB0000020|triage3|19:09
(integer) 1
127.0.0.1:6379> ZADD ed:queue 11793211120 WCB0000007|triage1|19:12
(integer) 1
127.0.0.1:6379> ZCARD ed:queue
(integer) 5
127.0.0.1:6379> ZRANGE ed:queue 0 -1
1) "WCB0000007|triage1|19:12"
2) "WCB0000003|triage2|19:07"
3) "WCB0000011|triage3|19:02"
4) "WCB0000020|triage3|19:09"
5) "WCB0000019|triage4|19:05"
# Next patient for the free cubicle (atomic pop - two nurses can never take the same patient):
127.0.0.1:6379> ZPOPMIN ed:queue
1) 1) "WCB0000007|triage1|19:12"
   2) "11793211120.0"
127.0.0.1:6379> ZPOPMIN ed:queue
1) 1) "WCB0000003|triage2|19:07"
   2) "21793210820.0"
127.0.0.1:6379> ZRANGE ed:queue 0 -1
1) "WCB0000011|triage3|19:02"
2) "WCB0000020|triage3|19:09"
3) "WCB0000019|triage4|19:05"

================================================================================
R4  CRITICAL LAB ALERTS - Redis Streams with a consumer group
================================================================================
127.0.0.1:6379> XGROUP CREATE lab:critical on-call-clinicians $ MKSTREAM
OK
# The laboratory system appends one entry per critical result:
127.0.0.1:6379> XADD lab:critical * mrn WCB0013656 test TROP value 52.000 resulted_at "2026-10-28 19:09"
"1793225809941-0"
127.0.0.1:6379> XADD lab:critical * mrn WCB0028022 test TROP value 57.000 resulted_at "2026-10-28 16:58"
"1793225809941-1"
127.0.0.1:6379> XADD lab:critical * mrn WCB0009591 test TROP value 58.000 resulted_at "2026-10-28 13:41"
"1793225809941-2"
127.0.0.1:6379> XLEN lab:critical
(integer) 3
# Dr Fairbanks' pager app reads new alerts for its consumer:
127.0.0.1:6379> XREADGROUP GROUP on-call-clinicians dr-fairbanks COUNT 2 STREAMS lab:critical >
   1793225809941-0  mrn=WCB0013656  test=TROP  value=52.000  resulted_at=2026-10-28 19:09
   1793225809941-1  mrn=WCB0028022  test=TROP  value=57.000  resulted_at=2026-10-28 16:58
# One alert is acknowledged after the call-back; the other stays pending until handled:
127.0.0.1:6379> XACK lab:critical on-call-clinicians 1793225809941-0
(integer) 1
127.0.0.1:6379> XPENDING lab:critical on-call-clinicians
1) "pending"
2) (integer) 1
3) "min"
4) "1793225809941-1"
5) "max"
6) "1793225809941-1"
7) "consumers"
8) 1) 1) "name"
      2) "dr-fairbanks"
      3) "pending"
      4) (integer) 1
# Unacknowledged alerts survive a consumer crash and can be re-assigned with XCLAIM.

================================================================================
```

The cache speed-up is modest (2×) on a lightly loaded local PostgreSQL that answers in a third of a millisecond. The real value of Redis is protecting the database from read storms and providing primitives (TTL locks, priority queues, streams) that are awkward in SQL. **Durability:** this instance runs with `appendonly no` (RDB snapshots only), which is acceptable for holds, caches and counters that can be rebuilt, but not for clinical data (R6).

3. Graph model — Cypher (`build_and_query_graph.py`)

The clinic's last three months (Aug–Oct 2026) as a property graph: 34,665 patients, 200 doctors, 26 conditions and 23 medications, connected by 59,495 `VISITED`, 76,618 `DIAGNOSED_WITH` and 70,356 `PRESCRIBED` relationships, plus `ALLERGIC_TO`, `WORKS_IN` and 14 curated `INTERACTS_WITH` drug-interaction edges (`interactions.csv`; illustrative, not for clinical use). Schema: `schema.cypher`. A Neo4j/AuraDB version of the load and queries is in `neo4j_version.cypher`.

Query	Question	Result
G1	Which patients hold both drugs of a *major* interacting pair?	Lisinopril+Losartan 189 patients, Sertraline+Sumatriptan 185, Warfarin+Ibuprofen 163
G2	Contact tracing: who shared a clinic (same doctor, same day) with a COVID-19 patient?	1,940 cases → 6,276 exposed patients, 10,320 exposure events
G2b	…and were themselves diagnosed within 14 days?	58 possible transmission links
G3	Which doctors share the most patients (referral network)?	Emergency Medicine ↔ Pediatrics tops the list
G4	Which conditions co-occur most (comorbidity)?	Hyperlipidemia + hypertension (1,035 patients)
G5	Shortest chain of shared clinicians linking two patients	Tyrone Jackson → Dr Raman → (patient) → Dr Okafor → Zara Ahmed: 4 hops
G6	Patients most similar to Tyrone Jackson by shared conditions	3 patients share all three of his codes

kuzu (Cypher) - medication interaction checks

```
G1  MEDICATION SAFETY - patients prescribed both drugs of a MAJOR interacting pair (Aug-Oct 2026)
-----------------------------------------------------------------------------------------------
MATCH (p:Patient)-[:PRESCRIBED]->(m1:Medication)-[i:INTERACTS_WITH]->(m2:Medication)<-[:PRESCRIBED]-(p)
WHERE i.severity = 'Major'
RETURN m1.name AS drug_1, m2.name AS drug_2, i.effect AS risk, count(DISTINCT p) AS patients
ORDER BY patients DESC
-----------------------------------------------------------------------------------------------
drug_1     | drug_2      | risk                                                | patients
-----------+-------------+-----------------------------------------------------+---------
Lisinopril | Losartan    | Dual RAAS blockade: hyperkalaemia and kidney injury | 189
Sertraline | Sumatriptan | Risk of serotonin syndrome                          | 185
Warfarin   | Ibuprofen   | Increased bleeding risk (anticoagulant + NSAID)     | 163
(3 rows, 40.3 ms)

===============================================================================================
G1b One patient's interaction alerts (any severity), with when each drug was prescribed
-----------------------------------------------------------------------------------------------
MATCH (p:Patient)-[r1:PRESCRIBED]->(m1:Medication)-[i:INTERACTS_WITH]->(m2:Medication)<-[r2:PRESCRIBED]-(p)
WITH p, count(i) AS alerts
ORDER BY alerts DESC, p.id LIMIT 1
MATCH (p)-[r1:PRESCRIBED]->(m1:Medication)-[i:INTERACTS_WITH]->(m2:Medication)<-[r2:PRESCRIBED]-(p)
RETURN p.mrn AS mrn, p.name AS patient, m1.name AS drug_1, r1.day AS rx_1, m2.name AS drug_2, r2.day AS rx_2,
       i.severity AS severity
ORDER BY severity, rx_2
-----------------------------------------------------------------------------------------------
mrn        | patient     | drug_1     | rx_1       | drug_2        | rx_2       | severity
-----------+-------------+------------+------------+---------------+------------+---------
WCB0023383 | Priya Davis | Sertraline | 2026-10-16 | Sumatriptan   | 2026-08-20 | Major
WCB0023383 | Priya Davis | Sertraline | 2026-08-04 | Sumatriptan   | 2026-08-20 | Major
WCB0023383 | Priya Davis | Warfarin   | 2026-10-16 | Sertraline    | 2026-08-04 | Moderate
WCB0023383 | Priya Davis | Warfarin   | 2026-08-04 | Sertraline    | 2026-08-04 | Moderate
WCB0023383 | Priya Davis | Warfarin   | 2026-10-16 | Acetaminophen | 2026-08-20 | Moderate
WCB0023383 | Priya Davis | Warfarin   | 2026-08-04 | Acetaminophen | 2026-08-20 | Moderate
WCB0023383 | Priya Davis | Warfarin   | 2026-10-16 | Acetaminophen | 2026-09-03 | Moderate
WCB0023383 | Priya Davis | Warfarin   | 2026-08-04 | Acetaminophen | 2026-09-03 | Moderate
WCB0023383 | Priya Davis | Warfarin   | 2026-10-16 | Sertraline    | 2026-10-16 | Moderate
WCB0023383 | Priya Davis | Warfarin   | 2026-08-04 | Sertraline    | 2026-10-16 | Moderate
(10 rows, 153.9 ms)

===============================================================================================
```

Rendered from graph_queries_output.txt (captured session output)

```
kuzu (Cypher) - contact tracing

G2  INFECTION CONTROL - contact tracing: who shared a clinic (same doctor, same day) with a COVID-19 patient?
-------------------------------------------------------------------------------------------------------
MATCH (c:Patient)-[d:DIAGNOSED_WITH]->(:Condition {code: 'U07.1'}),
      (c)-[v1:VISITED]->(doc:Doctor)<-[v2:VISITED]-(o:Patient)
WHERE v1.appointment_id = d.appointment_id AND v2.day = v1.day AND o.id <> c.id
RETURN count(DISTINCT c) AS covid_cases, count(DISTINCT o) AS exposed_patients, count(*) AS exposure_events
-------------------------------------------------------------------------------------------------------
covid_cases | exposed_patients | exposure_events
------------+------------------+----------------
1940        | 6276             | 10320
(1 rows, 63.7 ms)

=======================================================================================================
G2b Exposed patients who were themselves diagnosed with COVID-19 within 14 days (possible transmission)
-------------------------------------------------------------------------------------------------------
MATCH (c:Patient)-[d:DIAGNOSED_WITH]->(cov:Condition {code: 'U07.1'}),
      (c)-[v1:VISITED]->(doc:Doctor)<-[v2:VISITED]-(o:Patient)-[d2:DIAGNOSED_WITH]->(cov)
WHERE v1.appointment_id = d.appointment_id AND v2.day = v1.day AND o.id <> c.id
  AND d2.day > d.day AND d2.day <= d.day + INTERVAL('14 days')
RETURN c.mrn AS index_case, d.day AS exposure_day, doc.name AS clinic_of, o.mrn AS contact, d2.day AS contact_diagnosed
ORDER BY exposure_day
-------------------------------------------------------------------------------------------------------
index_case | exposure_day | clinic_of          | contact    | contact_diagnosed
-----------+--------------+--------------------+------------+------------------
WCB0036928 | 2026-08-06   | Dr Avery Patel     | WCB0030533 | 2026-08-20
WCB0043104 | 2026-08-07   | Dr Arjun Novak     | WCB0015819 | 2026-08-21
WCB0043791 | 2026-08-11   | Dr Alex Baker      | WCB0005078 | 2026-08-24
WCB0049931 | 2026-08-11   | Dr Ibrahim Silva   | WCB0032998 | 2026-08-14
WCB0014946 | 2026-08-12   | Dr Taylor Foster   | WCB0044263 | 2026-08-21
WCB0008765 | 2026-08-12   | Dr Taylor Foster   | WCB0044263 | 2026-08-21
WCB0007616 | 2026-08-13   | Dr Naomi Fairbanks | WCB0000772 | 2026-08-14
WCB0018240 | 2026-08-13   | Dr Naomi Fairbanks | WCB0000772 | 2026-08-14
WCB0037326 | 2026-08-18   | Dr Mei Owens       | WCB0045216 | 2026-08-20
WCB0039144 | 2026-08-19   | Dr Mei Baker       | WCB0004595 | 2026-08-27
WCB0041156 | 2026-08-19   | Dr Mei Baker       | WCB0004595 | 2026-08-27
WCB0047231 | 2026-08-20   | Dr Morgan Khan     | WCB0001465 | 2026-08-25
(58 rows, 112.7 ms)

=======================================================================================================

Rendered from graph_queries_output.txt (captured session output)
```

Graph vs SQL on the same questions. Run against the same data, both engines returned **identical** contact-tracing answers (1,940 / 6,276 / 10,320). That agreement was itself a useful check: an early Cypher version matched any visit that day rather than the diagnosing visit, and the SQL cross-check exposed it.

Question	Graph (Kùzu)	SQL (PostgreSQL)	Note
G2 contact tracing	**64 ms**	252 ms	SQL needs a self-join of appointment on doctor and date
G5 shortest path, 4 hops	**46 ms**, returns the path	184 ms, distance only	SQL needs `WITH RECURSIVE` breadth-first search (`shortest_path_recursive.sql`); returning the path itself is much harder

4. Column models

4.1 Columnar analytics — DuckDB on Parquet (executed)

401,731 lab results were extracted with their department and patient attributes into a ZSTD-compressed **Parquet** file. That is **5.7 MB**, against 35.7 MB as CSV, because each column is stored and compressed separately. The same two-year analytical query (monthly count, abnormal percentage and mean value per test per department, 3,331 groups):

Engine	Median
PostgreSQL, normalised operational schema (5-table join)	512 ms
PostgreSQL, same **denormalised** table (row store)	255 ms
DuckDB on Parquet (columnar, vectorised)	**80 ms**

On identical data the column store was **3.2× faster**; against the operational schema, 6.4×. The DuckDB plan shows why: the scan reads only the 5 projected columns out of 11 (3.6 MB of data).

```
duckdb - Parquet columnar analytics

Fact table: 401,731 lab results x 11 columns
  CSV extract             35.7 MB
  Parquet (ZSTD)           5.7 MB
  PostgreSQL lab_result table incl. indexes (fewer columns, normalised): 51.9 MB

Per-column compressed size inside the Parquet file (columnar storage + encoding):
  resulted_at         2542 KB   (uncompressed    3139 KB, ratio  1.2x)
  patient_id           689 KB   (uncompressed    3139 KB, ratio  4.6x)
  result_value         547 KB   (uncompressed     596 KB, ratio  1.1x)
  lab_result_id        432 KB   (uncompressed    3139 KB, ratio  7.3x)
  doctor_id            379 KB   (uncompressed     400 KB, ratio  1.1x)
  birth_year           346 KB   (uncompressed     348 KB, ratio  1.0x)
  test_code            218 KB   (uncompressed     223 KB, ratio  1.0x)
  department           145 KB   (uncompressed     199 KB, ratio  1.4x)
  sex                   87 KB   (uncompressed     100 KB, ratio  1.2x)
  abnormal_flag         76 KB   (uncompressed     100 KB, ratio  1.3x)
  priority              59 KB   (uncompressed     100 KB, ratio  1.7x)

Analytical query: monthly result count, abnormal % and mean value per test per department (2 years)
  PostgreSQL, normalised row store (5-table join):    512.4 ms  -> 3331 groups
  PostgreSQL, same denormalised table (row store):   254.6 ms
  DuckDB on Parquet (columnar, vectorised):           80.3 ms  -> 3331 groups
  columnar vs row store on identical data: 3.2x; vs the normalised operational schema: 6.4x

DuckDB plan - note the Parquet scan reads only the 5 projected columns of 11:
  ┌ Summary ──────────┐
  │ Total Time: 0.0535s │
  │ Data Read:   3.6 MB │
  └─────────────────────┘

  ┌ Projection ──────────────────────────────────────────────────────────────────────────────────────────────────────┐
  │ Projections: #0, #1, #2, results, abnormal_pct, mean_value                                                       │
  │ 3,331 rows                                                                                                 0µs │
  └──────────────────────────────────────────────────┬──────────────────────────────────────────────────────────────┘
  ┌ Hash Group By ──────────────────────────────────┴──────────────────────────────────────────────────────────────┐
  │ Groups: #0, #1, #2                                                                                           │
  │ Aggregates:                                                                                                  │
  │ count_star(), count_star() FILTER (WHERE #1) Filter: #1, avg(#4)                                             │
  │ 3,331 rows                                                                                              20.0ms │
  └──────────────────────────────────────────────────┬──────────────────────────────────────────────────────────────┘
  ┌ Projection ──────────────────────────────────────┴──────────────────────────────────────────────────────────────┐
  │ Projections: month, test_code, department, abnormal_flag != 'N', result_value                                │
  │ 401,731 rows                                                                                            70.0ms │
  └──────────────────────────────────────────────────┬──────────────────────────────────────────────────────────────┘
  ┌ Table Scan ──────────────────────────────────────┴──────────────────────────────────────────────────────────────┐
  │ Function: READ_PARQUET                                                                                       │
  │ Projections: resulted_at, test_code, department, abnormal_flag, result_value                                 │
  │ Total Files Read: 1                                                                                          │
  │ Filename(s):                                                                                                 │
  │ /home/claude/waynes-clinic-ads-portfolio/Portfolio-4-NoSQL-and-Advanced-Data-Models/columnar-duckdb/data/lab_fact.parquet │
  │ Row Groups Scanned: 4 / 4                                                                                    │
  │ 401,731 rows                                                                                            10.0ms │
  └──────────────────────────────────────────────────────────────────────────────────────────────────────────────┘

Sample output (General Medicine, HbA1c):
  2025-01  HBA1C  General Medicine  332  64.5  6.84
  2025-02  HBA1C  General Medicine  329  68.4  7.0
  2025-03  HBA1C  General Medicine  284  69.4  6.98
  2025-04  HBA1C  General Medicine  300  68.3  7.08
  2025-05  HBA1C  General Medicine  352  66.2  6.92
  2025-06  HBA1C  General Medicine  314  70.1  7.02

Rendered from columnar_duckdb_output.txt (captured session output)
```

4.2 Wide-column — Apache Cassandra (design, not executed)

`01_schema.cql` and `02_operations.cql` model the clinic **query-first**, with one table per access pattern and data duplicated on write:

- `appointments_by_doctor_day`, partition key `(doctor_id, clinic_date)`: the day list is one partition read.
- `slot_claims` plus `INSERT … IF NOT EXISTS`: a lightweight transaction (Paxos) gives compare-and-set booking.
- `patient_timeline`, partition `patient_id`, clustered by `event_time DESC`: history is already sorted on disk.
- `critical_results_by_day`: the lab worklist.

- `vitals_by_patient_day` with TimeWindowCompaction and a 30-day TTL: bedside telemetry at about 17,000 writes per patient per day, the workload Cassandra is built for.

`LOCAL_QUORUM` reads and writes on replication factor 3 give read-your-writes (R + W > N). The trade-offs are stated in the CQL: no joins, no ad-hoc filters (`ALLOW FILTERING` scans the cluster), and no cross-row rules such as "no overlapping appointments".

5. Suitability analysis

Requirement at Wayne's Clinic	Relational (PostgreSQL)	Document (MongoDB)	Key-value (Redis)	Graph	Wide-column (Cassandra)	Columnar (Parquet)
Book without double booking	**Best**: `EXCLUDE` range constraint, atomic	Partial: exact-slot unique index only	Helper: TTL hold	—	LWT on a fixed slot grid	—
Store diagnosis + prescription atomically, with allergy check	**Best**: one transaction, constraints and triggers	Good: multi-document transactions (replica set required)	—	—	Logged batch (atomic, not isolated)	—
Retrieve patient history	Good (1.25 ms)	**Best** (0.75 ms, one read, no joins)	Cache	—	Good (one partition)	—
Ad-hoc analytics / reporting	Good	Weak (1.1 s vs 0.2 s)	—	Relationship analytics	Weak	**Best**
Relationship questions (contact tracing, interactions, paths)	Possible, verbose (`WITH RECURSIVE`)	Weak	—	**Best**	—	—
Ephemeral, high-rate state (locks, queues, counters, alerts)	Possible	—	**Best**	—	—	—
Device telemetry ingest	Possible	Possible (time series)	—	—	**Best**	archive
Schema flexibility (e.g. HL7 FHIR resources, which are JSON)	`jsonb` columns	**Best**	—	—	—	—

Conclusion. No single model is best for a hospital. Clinical truth (bookings, diagnoses, prescriptions) needs strong consistency, multi-row constraints and serializable transactions, which is why **PostgreSQL remains the system of record** (Portfolios 1–3). The NoSQL stores earn their place as **purpose-built secondary stores**, fed from PostgreSQL by change-data-capture or batch ETL (diagram above):

- MongoDB for a FHIR-style patient portal and fast history reads.
- Redis for holds, queues and alerts.
- A graph database for infection control and medication-safety analytics.
- Parquet/DuckDB for management reporting.
- Cassandra only if the clinic adds inpatient telemetry.

This is *polyglot persistence*. Its cost is operational: more systems to secure for HIPAA, and copies that are only eventually consistent with the record (Portfolio 5 measures exactly that lag).

6. Reproduce

```
./run_portfolio4.sh      # needs PostgreSQL with Portfolios 1-2, MongoDB as replica set rs0, Redis, mongosh,
                         # Python: psycopg pymongo redis kuzu duckdb
```

Engines used in the build: MongoDB 8.0.23 (installed from conda-forge as a single-node replica set, which transactions require), Redis 7, Kùzu 0.11.3 and DuckDB 2.0.0 (Python packages). The graph and Parquet extracts are regenerated on each run and excluded from git.